Decidable Type-Checking for a Dependent Categorical Combinator Calculus

Lexi Aiello[1] and Eric Klavins[2]

[1]Department of Philosophy, University of Washington
[2]Electrical & Computer Engineering Department, University of Washington

April 14, 2026

Abstract

TODO: We present "DCC": a dependently-typed combinator calculus based on categorial combinators [4] with decidable type-checking, global confluence, and strong normalization of types. We achieve judgmental decidability by showing that the types of all objects can be expressed using a strongly-normalizing affine subset of the DCC. We show that this affine calculus is strongly normalizing by defining a decreasing measure under reduction. We also summarize a formalization of the theory and its properties in Lean 4.

1 Introduction

TODO: Simply-typed combinatory logics are known to be equivalent to the simply-typed λ-calculus [1]. These calculi can avoid contexts, and might enjoy benefits as such (e.g., unification). Work has been done on extending these logics with dependent types [2], but no implementation has been proposed, nor have such theories been formalized.

2 Background

2.1 The Untyped Categorical Combinators

The categorical combinators [4] are the basis of the Categorical Abstract Machine [3]. As shown by Cousineau et al., they are a potential compilation target for the λ-calculus. Categorical combinators simulate variable substitution with projection over nested tuples ("couples") constituting the "environment" and a curry combinator (Λ). The reduction rules for this calculus resemble the rules of the SK combinator calculus, and their categorical semantics are given by cartesian-closed categories [3, p. 178].

(ass)	$(x \circ y)z = x(yz)$
(fst)	$Fst(a, b) = a$
(snd)	$Snd(a, b) = b$
(dpair)	$\langle x, y\rangle z = (xz, yz)$
(curr)	$\Lambda(x)yz = x(y, z)$
(app)	$App(f, x) = fx$
(quote)	$`(x)y = x$

Figure 1: Reduction rules of the untyped combinators as seen in the Categorical Abstract Machine.

2.2 The Simply-Typed Categorical Combinators

Curien extends the combinators with type annotations, enabling simple typing. Quotation is omitted in the simply-typed variant, but it can be derived. The inference rules are as such:

$$\textsc{Id}\ \frac{\Gamma \vdash \alpha : \text{Type}}{\Gamma \vdash Id(\alpha) : \alpha \to \alpha} \qquad \textsc{Coup}\ \frac{\Gamma \vdash x : \alpha, y : \beta}{\Gamma \vdash (x, y) : \alpha \times \beta} \qquad \textsc{Comp}\ \frac{\Gamma \vdash x : \beta \to \gamma, y : \alpha \to \beta}{\Gamma \vdash (x \circ y) : \alpha \to \gamma}$$

$$\textsc{Fst}\ \frac{\Gamma \vdash \alpha : \text{Type}, \beta : \text{Type}}{\Gamma \vdash Fst(\alpha, \beta) : \alpha \times \beta \to \alpha} \qquad \textsc{Snd}\ \frac{\Gamma \vdash \alpha : \text{Type}, \beta : \text{Type}}{\Gamma \vdash Snd(\alpha, \beta) : \alpha \times \beta \to \beta} \qquad \textsc{Pair}\ \frac{\Gamma \vdash x : \alpha \to \tau_1, y : \alpha \to \tau_2}{\Gamma \vdash \langle x, y \rangle : \alpha \to \tau_1 \times \tau_2}$$

$$\textsc{Curr}\ \frac{\Gamma \vdash x : (\alpha \times \beta) \to \tau}{\Gamma \vdash \Lambda(x) : \alpha \to \beta \to \tau} \qquad \textsc{Appl}\ \frac{\Gamma \vdash \alpha : \text{Type}, \beta : \text{Type}}{\Gamma \vdash App(\alpha, \beta) : ((\alpha \to \beta) \times \alpha) \to \beta} \qquad \textsc{App}\ \frac{\Gamma \vdash f : \alpha \to \beta \qquad \Gamma \vdash x : \alpha}{\Gamma \vdash fx : \beta}$$

Figure 2: Inference rules of the simply-typed categorical combinators

3 Dependent Categorical Combinators

Our primary insights when extending the categorical combinators to dependent types are: when typed as Σ-pairs, couples can naturally encode contexts and typing judgments; pairs which produce couples correspond to substitution within contexts; the types of all primitive objects can be represented without the pair rule (the affine restriction) after adding a $Flip$ combinator. By showing that the affine restriction is strongly normalizing, we gain decidable judgmental equality ($\equiv$) and a decision procedure for type-checking.

As usual, we extend the calculus with a $\Pi(m, n)$ object. Arrows $\alpha \to \beta$ are represented by Π objects that quote their codomain. We re-interpret the base combinators as pertaining to Σ couples, instead of product couples.

3.1 Couples as Dependent Sums

We interpret couples (a, b) as Σ-pairs like such:

$$\textsc{Coup}_1\ \frac{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n \qquad \Gamma \vdash x : \alpha, y : \beta x}{\Gamma \vdash [\![\alpha, \beta]\!](a, b) : \Sigma(\alpha, \beta)}$$

Figure 3: Couples as dependent sums

We require that $\beta : \alpha \to \text{Sort } m$ be given explicitly in couples through the type annotation $[\![\alpha, \beta]\!]$. Similarly, we add type arguments to pairs ($[\![\beta, \gamma]\!]\langle f, g \rangle$) which allow the resulting couple's *snd* component to depend on the term fx. Pairs are not point-free, but couples are (with respect to x and y), since the categorical combinator $\Lambda(\ldots)$ can simulate a binder, but its type (and other combinators' types) must be bootstrapped with point-free couples. The point-free type of $[\![\alpha, \beta]\!](,)$ is available in the appendix.

This view of couples as sigma terms naturally encodes the notion of judgments. For example, the term $[\![\text{Sort } m, Id(\text{Sort } m)]\!](\alpha, x)$ corresponds to the judgment $x : \alpha$. Similarly, a context Γ can be encoded as a nested couple:

$$x_0 : A_0, x_1 : A_1(x_0), x_2 : A_2(x_0, x_1) \approx [\![\Sigma(A_0, A_1), A_2]\!]([\![A_0, A_1]\!](x_0, x_1), x_2) \tag{1}$$

In the categorical abstract machine, this encoding of judgments corresponds to an environment where variable assignments are explicitly typed.

3.2 Type Annotations as Points via Couple-Judgments

Except for Π, all combinator objects have type annotations specified as points. For example, the projection $Fst(\alpha, \beta)$ is indexed by terms denoting the type of the expected Σ-couple. This contradicts intuition that type arguments must be able to depend on other terms, and thus, cannot be points. However, by referring to terms in the context as the *snd* component of a Σ-couple ($\Sigma(\text{Sort } m, Id(\text{Sort } m))$) mentioning its type, they only need to be referred to by their respective sorts. As a result, we avoid another

potential duplication of type-points inside the combinators' types. Σ-judgments are also an example of "very dependent" types, as defined by Altenkirch et al. [2].

See (3.3.11) where we derive point-free $Id\ \alpha$ using the dependent curry (Λ) combinator and Σ-judgments.

3.3 Inference Rules

We make use of an infinite sort hierarchy to avoid paradoxes (i.e., Sort m : Sort $m+1$). We also use the notation $\Pi(x : A), B$, and may omit type arguments for legibility to denote an illustration of a type. However, the full combinatory, point-free types are accessible in the appendix. We use $A \to B$ as shorthand for our definition of arrows (5), but arrows are not first-class, and are merely an application of $\Pi(m, n)$.

3.3.1 Application and Π

The inference rule for function application is as usual, except the codomain β is applied the Σ-judgment $[\![\text{Sort } m, Id(\text{Sort } m)]\!](\alpha, x)$.

$$\frac{\Gamma \vdash f : \Pi(m,n)\alpha\beta \qquad \Gamma \vdash x : \alpha}{\Gamma \vdash fx : \beta[\![\text{Sort } m, Id(\text{Sort } m)]\!](\alpha, x)} \text{ App} \tag{2}$$

Figure 4: Inference rule: application

Since we do not make use of variables, $\Pi(m, n)$ is itself a dependent combinator, naively dependently typed (3). However, since we apply function type codomains (β) with $[\![\ldots]\!](\alpha, x)$, it can be expressed as an arrow type (4). Our use of the judgment-couple convention in the type of $\Pi(m, n)$ proves to be useful in several other combinator types, avoiding duplication of elements in the context.

$$\Pi(m,n) : \Pi(\alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n), \text{Sort } n \tag{3}$$

Figure 5: Naive typing of $\Pi(m, n)$ with a dependent type

$$\frac{}{\Gamma \vdash \Pi(m,n) : \text{Sort } m \to (\Sigma(\text{Sort } m, Id(\text{Sort } m)) \to \text{Sort } n) \to \text{Sort } n} \tag{4}$$

Figure 6: Arrow typing of $\Pi(m, n)$ using Σ-judgments

3.3.2 Arrow types

We define nondependent arrows ($\alpha \to \beta$) as such (where '(...) denotes the quotation combinator of arity 3):

$$\frac{\Gamma \vdash \alpha : \text{Sort } m \qquad \Gamma \vdash \beta : \text{Sort } n}{\Gamma \vdash \alpha \to \beta \coloneqq \Pi(m,n)\ \alpha\ \text{`}(\text{Sort } n, \Sigma(\text{Sort } n, Id(\text{Sort } m)), \beta)} \tag{5}$$

3.3.3 Couple Projections

We define dependent and nondependent versions of *Snd* projection to bootstrap several of the combinators' types. $Fst(\alpha, \beta)$ is not dependent, since α does not depend on β in $\Sigma(\alpha, \beta)$. We use $\circ'$ to denote nondependent composition where $g : \alpha \to \beta$ and $f : \beta \to \alpha$. The inference rules for projection on couples are as such:

$$\textsc{Fst}\ \frac{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n}{\Gamma \vdash Fst(\alpha, \beta) : \Sigma(\alpha, \beta) \to \alpha} \qquad \textsc{Snd'}\ \frac{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n}{\Gamma \vdash Snd'(\alpha, \beta) : \Sigma(\alpha, \beta) \to \beta}$$

$$\textsc{Snd}\ \frac{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n}{\Gamma \vdash Snd(\alpha, \beta) : \Pi(s : \Sigma(\alpha, \beta)), \beta(Snd\ s)}$$

3.3.4 Identity combinator

As do all other combinators, Id accepts α as a point. Its type is a simple arrow:

$$\textsc{Id}\ \frac{\Gamma \vdash \alpha : \text{Sort } m}{\Gamma \vdash Id(\alpha) : \alpha \to \alpha} \tag{6}$$

3.3.5 Flip combinator

We extend Curien's combinators [4] with a $Flip(x)$ combinator. We do so to avoid copying when applying an argument under $\Pi(m, n)$ as the domain of a subsequent $\Pi(n, o)\beta$ object:

$$\textsc{Flip}\ \frac{\Gamma \vdash x : \alpha \to \beta \to \gamma}{\Gamma \vdash Flip(x) : \beta \to \alpha \to \gamma} \tag{7}$$

3.3.6 Dependent and nondependent composition

We have nondependent composition as in Curien's combinators, and add a dependent version. Naively interpreting composition as dependent leads to the S combinator, so we use our Σ-couples to avoid duplication and preserve dependency.

$$\textsc{Naive comp.}\ \frac{\Gamma \vdash f : \Pi(x : \alpha), \beta(x), g : \Pi(x : ?), \gamma(fx)}{\Gamma \vdash (g \circ f) : \Pi(x : \alpha), \gamma(fx)} \tag{8}$$

Note that the codomain of f depends on $x : \alpha$, but the domain of g also depends on $fx : \beta(x)$. By duplicating x, ? can be determined:

$$\textsc{Naive comp.}\ \frac{\Gamma \vdash f : \Pi(x : \alpha), \beta(x), g : \Pi(x : \alpha, y : \beta(x)), \gamma x(fx)}{\Gamma \vdash (g \circ f) : \Pi(x : \alpha), \gamma(fx)} \tag{9}$$

We avoid copying x altogether by stipulating that f produce a couple $\Sigma(\alpha, \beta)$. As such, duplication is handed off to term-level β-reduction, and our types remain affine. We define composition point-free, up to the type arguments α, β, γ (10). The type of the nondependent composition operator is the same as in Curien's simply-typed combinators [4, p. 220] (11).

$$\textsc{Comp.}\ \frac{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n, \gamma : \Sigma(\alpha, \beta) \to \text{Sort } o}{\Gamma \vdash [\![\alpha, \beta, \gamma]\!] \circ : \Pi(f : \alpha \to \Sigma(\alpha, \beta), g : \Pi(x : \Sigma(\alpha, \beta)), \gamma(x), x : \alpha), \gamma(fx)} \tag{10}$$

Figure 7: Inference rule: composition in illustrative notation.

3.3.7 Pairs

Recall $\langle f, g \rangle x = (fx, gx)$. We extend the pair operator with type arguments β, γ, which dictate the types of the resulting fst, and snd component, respectively. All arguments, up to x, are given by points (12).

$$\frac{\text{Nondep. comp.}}{\Gamma \vdash f : \alpha \to \beta, g : \beta \to \gamma} \over \Gamma \vdash (g \circ f) : \alpha \to \gamma \tag{11}$$

Figure 8: Inference rule: nondependent composition

$$\frac{\text{Pair}}{\Gamma \vdash \beta : \text{Sort } n, \gamma : \beta \to \text{Sort } o, f : \alpha \to \beta, g : \Pi(x : \alpha), \gamma(fx)} \over \Gamma \vdash [\![\beta, \gamma]\!]\langle f, g\rangle : \alpha \to \Sigma(\beta, \gamma) \tag{12}$$

3.3.8 Quotation

The quotation operator ($`(\alpha, \beta, x)$) is not dependent, but a dependent version like the dependent K combinator can be derived as in our $Id(m)$ derivation (3.3.11).

$$\frac{\text{Quot}}{\Gamma \vdash \alpha : \text{Sort } m, \beta : \text{Sort } n, x : \alpha} \over \Gamma \vdash `(\alpha, \beta, x) : \beta \to \alpha \tag{13}$$

3.3.9 Tail and head-dependent Curry

Recall $\Lambda(x)yz = x(y, z)$. Since x is applied a Σ-couple, z must depend on y. That is, $z : \beta(y)$. However, x also depends on the couple $[\![\alpha, \beta]\!](y, z)$. To keep usages of y and z affine, we define two versions of $\Lambda(x)$. The former, $\Lambda'(\alpha, \beta, x)$ is "head-dependent", in that z may depend on y, but x has a fixed codomain (14). The latter, $\Lambda(\alpha, x)$ is "tail-dependent", in that y is itself a sort, z is an inhabitant of y, and x depends on $[\![\alpha, Id(\alpha)]\!](y, z)$ (15).

$$\frac{\text{Head Curry}}{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n, x : \Sigma(\alpha, \beta) \to \gamma} \over \Gamma \vdash \Lambda'(\alpha, \beta, x) : \Pi(x : \alpha, y : \beta(x)), \gamma \tag{14}$$

Figure 9: Inference rule: head-dependent curry

$$\frac{\text{Tail Curry}}{\Gamma \vdash \alpha : \text{Sort } m + 1, x : \Pi(s : \Sigma(\alpha, Id(\alpha))), \gamma(s)} \over \Gamma \vdash \Lambda(\alpha, x) : \Pi(x : \alpha, y : x), \gamma[\![\alpha, Id(\alpha)]\!](x, y) \tag{15}$$

Figure 10: Inference rule: tail-dependent curry

3.3.10 App combinator

As in Curien's combinators, the *App* combinator evaluates a couple as a computation. We extend *App* with type arguments ($App(n, \alpha, \beta)$) and give it the obvious dependent type (16).

$$\frac{\text{Appl}}{\Gamma \vdash \alpha : \text{Sort } m, \beta : \alpha \to \text{Sort } n} \over \Gamma \vdash App(n, \alpha, \beta) : \Pi(x : \Sigma(\Pi(x : \alpha), \beta(x), \alpha)), \beta(Snd\ x) \tag{16}$$

Figure 11: Inference rule for *App* written illustratively.

3.3.11 Derivation: Point-Free Id

Recall the types of Id, $\Lambda(\alpha, x)$, and $Snd(\alpha, \beta)$ as above. Note that the projection $Snd(\text{Sort } m, Id(\text{Sort } m))$ is well-typed, even if α in $Id(m)$ α is not known. So, let Id_m be as follows:

$$Id_m \coloneqq \Lambda(\text{Sort } m, Snd(\text{Sort } m, Id(\text{Sort } m)))$$

See the appendix for a mechanized proof that $Id_m(\alpha : \text{Sort } m)(x : \alpha)$ type-checks.

4 Reduction & Affine Reduction

Our reduction rules are the same as in the categorical abstract machine [3], except for *Flip*, and pairs, which carry type arguments:

$$\begin{array}{ll}
\text{(id)} & Id(\alpha)x \to x \\
\text{(ass')} & (x \circ' y)z \to x(yz) \\
\text{(ass)} & [\![\alpha, \beta, \gamma]\!] \circ xyz \to x(yz) \\
\text{(fst)} & Fst(\alpha, \beta)\ [\![\ldots]\!](a, b) \to a \\
\text{(snd)} & Snd(\alpha, \beta)\ [\![\ldots]\!](a, b) \to b \\
\text{(snd')} & Snd(\alpha, \beta)\ [\![\ldots]\!](a, b) \to b \\
\text{(curr')} & \Lambda'(\alpha, \beta, x)yz \to x\ [\![\alpha, \beta]\!](y, z) \\
\text{(curr)} & \Lambda(\alpha, x)yz \to x\ [\![\alpha, Id(\alpha)]\!](y, z) \\
\text{(app)} & App(n, \alpha, \beta)\ [\![\ldots]\!](f, x) \to fx \\
\text{(quote)} & `(\alpha, \beta, x)y \to x \\
\text{(flip)} & Flip(x)yz \to xzy \\
\text{(dpair)} & [\![\alpha, \beta]\!]\langle x, y\rangle\ z \to [\![\alpha, \beta]\!](xz, yz)
\end{array}$$

$$\text{(congr. fun.)}\quad \frac{f \to f'}{fx \to f'x} \qquad \text{(congr. arg.)}\quad \frac{x \to x'}{fx \to fx'}$$

Figure 12: Reduction rules of the dependently-typed categorical combinators

4.1 Affine Strong Normalization

Note that, excluding our *dpair* rule, each single-step combinator reduction uses each of its arguments at most once. That is, this restriction of single-step reduction is affine. Except for congruence, each rule decreases the number of combinators (we do not consider the couple $[\![\alpha, \beta]\!](,)$ to be a combinator). We thus define a syntactic measure of term size on objects in our calculus, and show that it decreases under affine reduction ($\dashrightarrow$) to show that the affine reduction is strongly normalizing.

Formally, we define affine reduction ($\dashrightarrow$) as such:

```
inductive RedAff : Expr → Expr → Prop
  | app      : RedAff (DCC (App(_n, _δ, _τ) ⟦α, β⟧(f, x))) (DCC (f x))
  | fst      : RedAff (DCC (Fst(_δ, _τ) ⟦α, β⟧(a, b))) a
  | snd      : RedAff (DCC (Snd(_δ, _τ) ⟦α, β⟧(a, b))) b
  | snd'     : RedAff (DCC (Snd'(_δ, _τ) ⟦α, β⟧(a, b))) b
  | flip     : RedAff (DCC (Flip(e) x y)) (DCC (e y x))
  | id       : RedAff (DCC (Id(_α) x)) x
  | quot     : RedAff (DCC (`(_α, _β, x) y)) x
  | comp     : RedAff (DCC ((⟦α, β, γ⟧∘ x y) z)) (DCC (x (y z)))
  | comp'    : RedAff (DCC ((x ∘' y) z)) (DCC (x (y z)))
  | curr     : RedAff (DCC (Λ(α, x) y z)) (DCC (x ⟦α, Id(α)⟧(y, z)))
  | curr'    : RedAff (DCC (Λ'(α, β, x) y z)) (DCC (x ⟦α, β⟧(y, z)))
  | congrFun : RedAff f f' → RedAff (DCC (f x)) (DCC (f' x))
  | congrArg : RedAff x x' → RedAff (DCC (f x)) (DCC (f x'))
```

We define the size of all reducible combinators to be the successor of the sum of the sizes of the combinators' constituent term points. For example, $Id(\alpha)$ has size 1, since its point is a term-type, not

```
inductive Size : Expr → ℕ → Prop
  | sig    : Size (DCC Σ(_m, _n)) 0
  | coup   : Size (DCC ⟦α, β⟧(,)) 0
  | pi     : Size (DCC Π(_m, _n)) 0
  | sort   : Size (DCC Sort m) 0
  | id     : Size (DCC Id(α)) 1
  | comp   : Size (DCC ⟦α, β, γ⟧∘) 1
  | fst    : Size (DCC Fst(α, β)) 1
  | snd    : Size (DCC Snd(α, β)) 1
  | snd'   : Size (DCC Snd'(α, β)) 1
  | appl   : Size (DCC App(_n, α, β)) 1
  | flip   : Size e m → Size (DCC Flip(e)) m.succ
  | quot   : Size x m → Size (DCC `(α, β, x)) m.succ
  | comp'  : Size f m → Size g n → Size (DCC (f ∘' g)) (m + n).succ
  | pair   : Size f m → Size g n → Size (DCC ⟦α, β⟧⟨f, g⟩) (m + n).succ
  | curr   : Size x m → Size (DCC Λ(α, x)) m.succ
  | curr'  : Size x m → Size (DCC Λ'(α, β, x)) m.succ
  | app    : Size f m → Size x n → Size (DCC (f x)) (m + n)
```

Figure 13: Syntactic size measure for the combinators

a term with computational relevance. Irreducible terms are assigned a size of 0 (13). We also define a computable `size (e : Expr) : ((n : ℕ) × (PLift (Size e n)))` function which produces a proof of `Size e n`, thus gaining soundness by default (definition omitted, see appendix).

Theorem 1. *If $A \in Expr$ and $A \dashrightarrow B$, then $size(B) < size(A)$.*

Proof.

```
theorem RedAff.decreasing (h : RedAff e e') : (size e').fst < (size e).fst := by
  induction h <;> (simp [size]; try grind)
```

□

We define strong normalization using Lean's accessibility predicate, and use our decreasing measure to produce a proof of `Acc (fun b a => RedAff a b) e`.

Theorem 2. *If $A \in Expr$, then A is strongly-normalizing under affine reduction ($\dashrightarrow$).*

Proof.

```
def sn (e : Expr) : Acc (fun b a => RedAff a b) e := Acc.intro e (fun e' hr => by
  have h := hr.decreasing
  exact e'.sn)
termination_by e.size.fst

instance instWellFounded : WellFounded (fun a b => RedAff b a) := ⟨sn⟩
```

□

4.2 Affine Global Confluence

We define the transitive ($\dashrightarrow^+$) and reflexive transitive closure ($\dashrightarrow^*$) like such:

```
def RedAffTrans     := Relation.TransGen RedAff
def RedAffReflTrans := Relation.ReflTransGen RedAff
```

Theorem 3. *For all $A \in Expr$, A is confluent under $\dashrightarrow$.*

Proof. We proceed by induction and case analysis on $\dashrightarrow$. Cases have been omitted for brevity. Consult the appendix.

```
theorem RedAff.conf (h₁ : RedAff A B) (h₂ : RedAff A C) :
  ∃ D, RedAffReflTrans B D ∧ RedAffReflTrans C D := by
  induction h₁ generalizing C with
  | comp =>
    cases h₂ with
    | comp => exact ⟨_, .refl, .refl⟩
    | congrFun h =>
      cases h with
```

```
      | congrFun h =>
        cases h with
        | congrFun h => cases h
        | congrArg h => exact ⟨_, .single (.congrFun h), .single .comp⟩
      | congrArg h => exact ⟨_, .single (.congrArg (.congrFun h)), .single .comp⟩
      | congrArg h =>
        exact ⟨_, .single (.congrArg (.congrArg (by assumption))), .single .comp⟩
  | congrFun h_r ih =>
    cases h₂
    case comp' | quot | id | app | fst | snd | snd' => cases h_r
    case flip =>
      cases h_r with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single .flip, .single (.congrArg h)⟩
    case comp =>
      cases h_r with
      | congrFun h =>
        cases h with
        | congrFun h => cases h
        | congrArg h => exact ⟨_, .single .comp, .congrFun (.single h)⟩
      | congrArg h => exact ⟨_, .single .comp, .congrArg (.congrFun (.single h))⟩
    case curr =>
      cases h_r with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single .curr, .single (.congrArg (.sigl (by assumption)))⟩
    case curr' =>
      cases h_r with
      | congrFun h => cases h
      | congrArg h => exact ⟨_, .single .curr', .single (.congrArg (.sigl (by assumption)))⟩
    case congrFun f f' x f'' h =>
      have ⟨_, h₁, h₂⟩ := ih h
      exact ⟨_, .congrFun h₁, .congrFun h₂⟩
    case congrArg f f' x x'' h_x =>
      exact ⟨_, .congrArg (.single h_x), .congrFun (.single h_r)⟩
  | _ =>
    -- remaining cases omitted
    sorry
```

□

Theorem 4. *For all $A \in Expr$, A is confluent under $\dashrightarrow^*$.*

Proof. We proceed by induction on the well-founded relation $\dashrightarrow$ (4.1).

```
theorem RedAffReflTrans.conf (h₁ : RedAffReflTrans A B) (h₂ : RedAffReflTrans A C) :
  ∃ D, RedAffReflTrans B D ∧ RedAffReflTrans C D := by
  induction A using instWellFounded.induction generalizing B C with
  | h _ ih =>
    match h₁.cases_head, h₂.cases_head with
    | .inr ⟨_, hra, hrc⟩, .inr ⟨_, hra', hrc'⟩ =>
      have ⟨_, c₁', c₂'⟩ := hra.conf hra'
      have ⟨_, hrB, hrB'⟩ := ih _ hra hrc c₁'
      have ⟨_, hrC, hrC'⟩ := ih _ hra' hrc' c₂'
      have ⟨_, h₁, h₂⟩ := ih _ hra (c₁'.trans hrB') (c₁'.trans hrC')
      exact ⟨_, .trans hrB h₁, .trans hrC h₂⟩
    | .inl heq₁, .inl heq₂ =>
      rw [← heq₁, ← heq₂]
      exact ⟨_, .refl, .refl⟩
    | .inl heq₁, .inr _ =>
      rw [← heq₁]
      exact ⟨_, h₂, .refl⟩
    | .inr _, .inl heq₁ =>
      rw [← heq₁]
      exact ⟨_, .refl, h₁⟩
```

□

Theorem 5. *For all $A \in Expr$, if $A \dashrightarrow B$ and $A \dashrightarrow C$, and B and C are normal forms, then $B = C$.*

Proof. We define normal forms as such:

```
def IsNormAff (e : Expr) : Prop := ¬ ∃ e', RedAff e e'
```

We proceed by induction on the common reduct of B and C:

```
theorem RedAff.norm_unique (h₁ : RedAff A B) (h_norm : B.IsNormAff)
  (h₂ : RedAff A C) (h_norm₂ : C.IsNormAff) : B = C := by
  have ⟨D, ha, hb⟩ := h₁.conf h₂
  match ha.cases_head, hb.cases_head with
  | .inl h₁, .inl h₂ =>
    simp_all
  | .inr ⟨c₁, h₁, h₂⟩, _ =>
    exact h_norm.elim ⟨_, h₁⟩
  | _, .inr ⟨c₁, h₁, h₂⟩ =>
    exact h_norm₂.elim ⟨_, h₁⟩
```

□

Theorem 6. *For all $A \in Expr$, if $A \dashrightarrow^* B$ and $A \dashrightarrow^* C$, and B and C are normal forms, then $B = C$.*

Proof. We proceed by induction on the common reduct of B and C.

```
theorem RedAffReflTrans.norm_unique (h₁ : RedAffReflTrans A B) (h_norm : B.IsNormAff)
  (h₂ : RedAffReflTrans A C) (h_norm₂ : C.IsNormAff) : B = C := by
  have ⟨D, ha, hb⟩ := h₁.conf h₂
  match ha.cases_head, hb.cases_head with
  | .inl h₁, .inl h₂ =>
    simp_all
  | .inr ⟨c₁, h₁, h₂⟩, _ =>
    exact h_norm.elim ⟨_, h₁⟩
  | _, .inr ⟨c₁, h₁, h₂⟩ =>
    exact h_norm₂.elim ⟨_, h₁⟩
```

□

4.3 Decidable Beta-Equality

We give a decision procedure for $=_\beta$. We do so by proving soundness and completeness for β-reduction (with respect to normal-forms). We thus implement an instance of:

```
∃ C, RedAffReflTrans A C ∧ RedAffReflTrans B C ∧ C.IsNormAff
```

First, we define `step` and `reduce` functions with soundness using our measure:

```
def step? (e : Expr) : Option ((e' : Expr) × (PLift (RedAff e e'))) :=
  match e with
  | (DCC (App(_n, _δ, _τ) ⟦_α, _β⟧(f, x)))   => .some ⟨(DCC (f x)), ⟨.app⟩⟩
  | (DCC (Fst(_δ, _τ) ⟦_α, _β⟧(a, _b)))      => .some ⟨a, ⟨.fst⟩⟩
  | (DCC (Snd(_δ, _τ) ⟦_α, _β⟧(_a, b)))      => .some ⟨b, ⟨.snd⟩⟩
  | (DCC (Snd'(_δ, _τ) ⟦_α, _β⟧(_a, b)))     => .some ⟨b, ⟨.snd'⟩⟩
  | (DCC (Flip(e) x y))                      => .some ⟨(DCC (e y x)), ⟨.flip⟩⟩
  | (DCC (Id(_α) x))                         => .some ⟨x, ⟨.id⟩⟩
  | (DCC (`(_α, _β, x) _y))                  => .some ⟨x, ⟨.quot⟩⟩
  | (DCC ((⟦_α, _β, _y⟧∘ x y) z))            => .some ⟨(DCC (x (y z))), ⟨.comp⟩⟩
  | (DCC ((x ∘' y) z))                       => .some ⟨(DCC (x (y z))), ⟨.comp'⟩⟩
  | (DCC (Λ(α, x) y z))                      => .some ⟨(DCC (x ⟦α, Id(α)⟧(y, z))), ⟨.curr⟩⟩
  | (DCC (Λ'(α, β, x) y z))                  => .some ⟨(DCC (x ⟦α, β⟧(y, z))), ⟨.curr'⟩⟩
  | DCC (f x) =>
    match f.step?, x.step? with
    | .some ⟨f', ⟨h₁⟩⟩, _ => .some ⟨(DCC (f' x)), ⟨.congrFun h₁⟩⟩
    | _, .some ⟨x', ⟨h₁⟩⟩ => .some ⟨(DCC (f x')), ⟨.congrArg h₁⟩⟩
    | _, _ => .none
  | _ => .none

def reduce (e : Expr) : Expr :=
  match step? e with
  | .some ⟨e', ⟨h⟩⟩ =>
    have _h_size : e'.size.fst < e.size.fst := h.decreasing
    e'.reduce
  | .none => e
```

Theorem 7. *For all $A \in Expr$, and `reduce A = B`, then $A \dashrightarrow^* B$.*

Proof.

```
theorem reduce_sound (heq : reduce A = B) : RedAffReflTrans A B := by
  induction A using instWellFounded.induction generalizing B with
  | h a ih =>
    unfold reduce at heq
    split at heq
    case h_1 h_red h_step_eq =>
      exact Relation.ReflTransGen.trans (.single h_red) (ih _ h_red heq)
    case h_2 => simp_all only [forall_eq']; exact .refl
```

□

By using our decreasing measure and showing completeness for `step?` with respect to normal forms under `-->`, we prove completeness of `reduce` with respect to normal forms under `-->*`.

Lemma 1. *For all $e \in Expr$, if e is a normal form, then `step? e = none`.*

Proof.

```
  theorem IsNormAff.complete (h : A.IsNormAff) : (step? A) = .none := by
  apply Option.eq_none_iff_forall_ne_some.mpr
  intro ⟨e', ⟨h₂⟩⟩ heq
  exact (h ⟨_, h₂⟩)
```

□

Theorem 8. *For all $A \in Expr$, if A `-->` B and B is a normal form, then `step A = some B`.*

Proof. We proceed by case analysis with respect to the unfolding of `step?`:

```
theorem RedAff.complete (h₁ : RedAff A B) (h_norm : B.IsNormAff) :
  (step? A) = .some ⟨B, ⟨h₁⟩⟩ := by
  unfold step?
  split
  case h_4 | h_3 | h_2 | h_1  =>
    cases heq : h₁
    · rfl
    · rename_i h_r
      have h := h_r.complete h_norm.congrFun
      simp [step?] at h
    · rename_i h_r
      cases h_r
      · rename_i h_r
        cases h_r
        · rename_i h; cases h
        · have h := h_norm.complete
          simp [step?] at h
      · have h := h_norm.complete
        simp [step?] at h
  case h_7 | h_6 =>
    cases heq : h₁
    · rfl
    · rename_i h; cases h
    · have h := h_norm.complete
    simp [step?] at h
  case h_5 | h_8 | h_9 | h_10 | h_11 | h_12 | h_13 =>
    sorry -- omitted for brevity. See appendix.
```

□

Lemma 2. *For all $e \in Expr$, if e is a normal form, then `reduce e = e`.*

Proof.

```
theorem IsNormAff.complete_reduce (h : A.IsNormAff) : reduce A = A := by
  unfold reduce
  rw [h.complete]
```

□

Theorem 9. *For all $A \in Expr$, if A `-->*` B, and B is a normal form, then `reduce A = B`.*

Proof. By well-founded recursion with respect to the decreasing measure `size`:

```
theorem RedAffReflTrans.complete (h₁ : RedAffReflTrans A B) (h_norm : B.IsNormAff) :
  reduce A = B := by
  match h₁.cases_head with
  | .inl h =>
    simp_all
    exact h_norm.complete_reduce
  | .inr ⟨c, h₁, h₂⟩ =>
    have _h : sizeOf c < sizeOf A := h₁.decreasing
    have h := RedAffReflTrans.complete h₂ h_norm
    rw [h₁.reduce_same, h]
termination_by A.size.fst
```

□

Finally, we show $=_\beta$ is decidable.

Theorem 10. *For all $A \in Expr$ and $B \in Expr$, $(\exists C, A \dashrightarrow^* C \wedge B \dashrightarrow^* C \wedge IsNorm(C)) \vee \neg(\exists C, A \dashrightarrow^* C \wedge B \dashrightarrow^* C IsNorm(C))$.*

Proof. This follows from completeness and soundness of `reduce`:

```
instance : Decidable (∃ C, RedAffReflTrans A C ∧ RedAffReflTrans B C ∧ C.IsNormAff) :=
  if heq : reduce A = reduce B then .isTrue
    ⟨_, reduce_sound rfl, reduce_sound heq.symm, reduce_norm rfl⟩
  else
    .isFalse (fun ⟨C, h₁, h₂, h_norm⟩ => by
      have ha := h₁.complete h_norm
      have hb := h₂.complete h_norm
      simp_all)
```

□

5 Type Inference

Note that the types of all of the dependent categorical combinators are affine (3.3). Hence, we use our decision procedure (4.3) for $=_\beta$ under affine-reduction to prove decidability of type-checking.

We do not normalize inside our judgment relation, nor do we have a conversion case. We bake conversion into each applicable constructor with a `Reduces` predicate by normalizing the expected and actual types a la carte:

```
inductive Reduces : Expr → Expr → Type
  | mk : RedAffReflTrans A B → IsNormAff B → Reduces A B
deriving Repr

theorem Reduces.complete : Reduces A B → reduce A = B :=
  fun ⟨ha, hb⟩ => ha.complete hb

def reduce_reduces (h : reduce A = B) : Reduces A B := ⟨reduce_sound h, reduce_norm h⟩

def reduce_reduces' : Reduces A A.reduce := reduce_reduces rfl
```

For example, application is inferred like such:

```
inductive Judge : Expr → Expr → Type where
  | app   : Judge f t_f
    → Reduces t_f (DCC (Π(m, n) α β))
    → Judge x t_x
    → Reduces t_x α
    → Judge (DCC (f x)) (DCC (β ⟦Sort m, Id(Sort m)⟧(α, x)))
```

We defer full type normalization to a supplementary predicate, `JudgeConv`:

```
inductive JudgeConv : Expr → Expr → Type
  | mk : Judge x A → Reduces A B → JudgeConv x B
```

We define an `IsArrow` predicate for convenience in pattern matching arrows, which are by convention, Π-objects containing quotation in their codomain. We do not assume Π-extensionality.

```
inductive IsArrow : Expr → (α β : Expr) → Level → Level → Type
  | mk    : Reduces A (arrow α β m n) → IsArrow A α β m n
deriving Repr

def is_arrow (t : Expr) : Option ((α β : Expr) × (m n : Level) × IsArrow t α β m n) :=
  match heq₀ : t.reduce with
  | DCC (Π(m, n) α `(Sort n₂, Σ(Sort m₂, Id(Sort m₃)), β)) =>
    if heq : m = m₂ ∧ m = m₃ ∧ n = n₂ then by
      have ⟨ha, hb, hc⟩ := heq
      rw [← ha, ← hb, ← hc] at heq₀
      refine .some ⟨α, β, m, n, ?_, ?_⟩
      · exact reduce_sound heq₀
      · exact reduce_norm heq₀
    else .none
  | _ => .none
```

Our `Judge` relation models the inference rules (3.3). We have omitted the individual types and corresponding constructors for brevity, but give the constructor for tail-dependent Curry (15) and App (16) as examples. See the appendix and supplementary materials for the full formalization.

```
def curr.type (α γ : Expr) (m n : Level) : Expr :=
  (DCC (Π(m.succ, n) α ((Flip(Π(m, n)) (γ ∘'
  (Λ'(Sort m.succ, Id(Sort m.succ), Id(Σ(Sort m.succ, Id(Sort m.succ))))
    ⟦Sort m.succ, Id(Sort m.succ)⟧(α, Id(α))))) ∘' Snd'(Sort m.succ, α))))

def app_e.type (α β : Expr) (m n : Level) : Expr :=
  let t_f := DCC (Π(m, n) α β)
  let t_x := DCC `(Sort n, t_f, α)

  DCC (Π(max m n, n)
      Σ(t_f, t_x)
      ((Λ'(Sort m, Id(Sort m), β) α) ∘' (Snd(t_f, t_x) ∘ Snd'(Sort (max m n), Σ(t_f, t_x)))))

inductive Judge : Expr → Expr → Type where
  | curr  : Judge α t_α
    → Reduces t_α (DCC (Sort m.succ))
    → Judge x t_x
    → Reduces t_x (DCC (Π(m.succ, n) Σ(α, Id(α)) γ))
    → Judge (DCC Λ(α, x)) (curr.type α γ m n)
  | app_e : Judge α t_α
    → Reduces t_α (DCC Sort m)
    → Judge (DCC App(n, α, β)) (app_e.type α β m n)
  | app   : Judge f t_f
    → Reduces t_f (DCC (Π(m, n) α β))
    → Judge x t_x
    → Reduces t_x α
    → Judge (DCC (f x)) (DCC (β ⟦Sort m, Id(Sort m)⟧(α, x)))
  | sort  : Judge (DCC Sort m) (DCC Sort m.succ)
deriving Repr
```

5.1 Computing Type Inference

Using our terminating affine β-reduction function and `reduce_reduces`, we define an `infer` function with soundness. We give only the cases described in our `Judge` illustration, though the rest are available in supplementary materials. Pattern matching against our `Judge` constructors is somewhat obtuse, primarily due to our design decision to avoid adding a conversion constructor and reducing the inferred types.

```
def infer (e : Expr) : Option ((t : Expr) × (Judge e t)) :=
  match e with
  | DCC Sort m => .some ⟨DCC Sort m.succ, .sort⟩
  | DCC App(n, α, β) =>
    match α.infer with
    | .some ⟨t_α, h_t_α⟩ =>
      match heq₀ : t_α.reduce with
      | DCC Sort m =>
        .some ⟨_, .app_e h_t_α (reduce_reduces heq₀)⟩
      | _ => .none
    | _ => .none
  | DCC Λ(α, x) =>
    match heq₀ : α.infer, x.infer with
    | .some ⟨t_α, h_t_α⟩, .some ⟨t_x, h_t_x⟩ =>
```

```
      match heq₁ : t_α.reduce with
      | DCC Sort m + 1 =>
        match heq₃ : t_x.reduce with
        | (DCC (Π(m₂ + 1, n) Σ(α₁, Id(α₂)) y₂)) =>
          if heq₂ : m₂ = m ∧ α₁ = α ∧ α = α₂ then
            have ⟨ha, hb, hc⟩ := heq₂
            .some ⟨_, Judge.curr
              h_t_α
              (reduce_reduces heq₁)
              h_t_x
              (ha ▸ hb ▸ hc ▸ (reduce_reduces heq₃))⟩
          else .none
        | _ => .none
      | _ => .none
    | _, _ => .none
  | DCC (f x) => match f.infer, x.infer with
    | .some ⟨t_f, h_f⟩, .some ⟨t_x, h_x⟩ =>
      match heq₀ : t_f.reduce, heq₁ : t_x.reduce with
      | DCC (Π(m, n) α₁ β), α₂ =>
        if heq : α₁ = α₂ then
          .some ⟨(DCC (β ⟦Sort m, Id(Sort m)⟧(α₁, x))),
            .app h_f (reduce_reduces heq₀)
                 h_x (reduce_reduces (heq ▸ heq₁))⟩
        else .none
      | _, _ => .none
    | _, _ => .none
  | _ => sorry -- remaining cases omitted for brevity

def infer_conv (e : Expr) : Option ((t : Expr) × JudgeConv e t) :=
  match infer e with
  | .some ⟨t, h⟩ =>
    .some ⟨t.reduce, ⟨h, reduce_reduces rfl⟩⟩
  | _ => .none
```

Avoiding a conversion constructor proves to be quite advantageous, since neither confluence or induction are necessary, and case analysis suffices to prove useful properties. For example, a proof of preservation of $Id(\alpha)x$ is quite straightforward.

Theorem 11. *For all $\alpha \in Expr$ and $x \in Expr$, if $Id(\alpha)\ x : reduce(\alpha)$, then $x : reduce(\alpha)$.*

Proof. We proceed by case analysis on the judgment $Id(\alpha)\ x : reduce(\alpha)$.

```
def id_preservation (h : JudgeConv (DCC (Id(α) x)) α.reduce) : (JudgeConv x α.reduce) :=
  match h with
  | ⟨.app (.id _ _) b h_t_x _, h_red⟩ => by
    cases reduce_arrow b.complete
    exact ⟨h_t_x, by assumption⟩
```

□

5.1.1 Infer Completeness

Yet another advantage of avoiding a conversion constructor, and of having decidable $\beta_{=}$ is that completeness is provable. We have omitted all but the `curr`, `app_e`, `sort`, and `app` constructors (see supplementary materials).

Theorem 12. *For all $e \in Expr$ and $t \in Expr$, if $e : t$, then `infer e = some t`.*

Proof. We proceed by parallel induction on the typing derivation and case analysis on the unfolding of `infer`.

```
theorem Judge.complete (h_t : Judge e t) : infer e = .some ⟨t, h_t⟩ := by
  induction h_t with
  | sort =>
    unfold infer
    rfl
  | curr h_t_α h_red₁ h_t_x h_red₂ ih₁ ih₂ =>
    have ha := h_red₁.complete
    have hb := h_red₂.complete
    unfold infer
    rw [ih₁, ih₂]
    simp
```

```
    split
    case h_1 heq =>
      rw [heq] at ha
      cases ha
      split
      case h_1 heq =>
        rw [heq] at hb
        cases hb
        simp
        split
        rfl
      case h_2 h =>
        exact (h _ _ _ _ _ hb).elim
    case h_2 h =>
      exact (h _ ha).elim
  | app_e h_t_α h_red₁ ih =>
    have ha := h_red₁.complete
    unfold infer
    rw [ih]
    simp
    split
    case h_1 heq =>
      rw [heq] at ha
      cases ha
      rfl
    case h_2 h =>
      exact (h _ ha).elim
  | app h_t_f h_red_f h_t_x h_red_x ih₁ ih₂ =>
    have ha := h_red_f.complete
    have hb := h_red_x.complete
    unfold infer
    simp [ih₁, ih₂]
    split
    case h_1 heq =>
      rw [heq] at ha
      cases ha
      cases hb
      simp
      aesop
    case h_2 h =>
      exact (h _ _ _ _ ha).elim
```

□

5.2 Judgmental Decidability

Using our proof of completeness and soundness proof produced by `infer`, we demonstrate judgmental decidability.

Theorem 13. *For all $e \in Expr$ and $t \in Expr$, $e : t \vee \neg e : t$.*

Proof.

```
def Judge.decide : Judge e t ⊕ (Judge e t → PLift False) :=
  if h : (infer e).isSome then
    if heq : ((infer e).get h).fst = t then
      .inl (heq ▸ ((infer e).get h).snd)
    else
      .inr fun hn => by
        have h_comp := hn.complete
        simp_all
  else .inr fun hn => by
    have h_comp := hn.complete
    simp_all

def JudgeConv.decide : JudgeConv e t ⊕ (JudgeConv e t → PLift False) :=
  if h : (infer e).isSome then
    if heq : ((infer e).get h).fst.reduce = t then
      .inl ⟨((infer e).get h).snd, reduce_reduces heq⟩
    else .inr fun ⟨ha, hb⟩ => by
      have h := ha.complete
      simp_all
      exact (heq hb.complete).elim
  else .inr fun ⟨ha, hb⟩ => by
```

```
have h := ha.complete
simp_all
```

□

References

[1] Thorsten Altenkirch et al. "Combinatory Logic and Lambda Calculus Are Equal, Algebraically". In: *International Conference on Formal Structures for Computation and Deduction*. 2023. URL: https://api.semanticscholar.org/CorpusID:259298739.

[2] Thorsten Altenkirch et al. "The Münchhausen Method in Type Theory". In: *28th International Conference on Types for Proofs and Programs (TYPES 2022)*. Ed. by Delia Kesner and Pierre-Marie Pédrot. Vol. 269. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 10:1–10:20. ISBN: 978-3-95977-285-3. DOI: 10.4230/LIPIcs.TYPES.2022.10. URL: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2022.10.

[3] G. Cousineau, P-L. Curien, and M. Mauny. "The categorical abstract machine". In: *Functional Programming Languages and Computer Architecture*. Ed. by Jean-Pierre Jouannaud. Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 50–64. ISBN: 978-3-540-39677-2.

[4] P.-L. Curien. "Categorical combinators". In: *Information and Control* 69.1 (1986), pp. 188–254. ISSN: 0019-9958. DOI: https://doi.org/10.1016/S0019-9958(86)80047-X. URL: https://www.sciencedirect.com/science/article/pii/S001999588680047X.